

ZLC FPGA Pulse Streamer 测试报告

4ch 仿真 / Vivado evidence / 40ch GUI+server 自查

Vivado 2019.x 路径、runtime edge table、Pulse GUI 前端和真实硬件上机
流程

2026-06-02

Contents

1	结论先行	1
2	架构边界	3
3	Vivado 与路径策略	4
4	4ch 仿真证据	5
5	4ch Vivado build 证据	7
6	40ch 自查边界	9
7	Server 与 GUI 自查	11
8	上机前 checklist	13
9	故障定位	14

1

结论先行

本报告记录当前仓库中 FPGA pulse-streamer、40-channel pulse preset、server/API 和 Pulse GUI 的本地自查结果。报告的目的不是替代真实 FPGA 上电检查，而是在去 Verilog 电脑之前，把会在电脑侧失败的部分尽量提前排掉：Vivado 工具链、HDL 仿真、4ch bitstream/timing artifact、40ch runtime program 生成、server boundary、Pulse GUI 前端调用链，以及手册里的运行步骤。

Check	Result	Evidence
Vivado discovery	PASS	vivado_env.bat found local Vivado 2019.1
4ch xsim	PASS	ZLC_SIM_PASS in xsim.log
4ch bit/LTX	PASS	bit and ltx artifacts exist
4ch timing	PASS	WNS 0.759 ns; timing constraints met=True
40ch preset compile	PASS	40 channels, 6 edges, final mask 0
40ch dry-run Tcl	PASS	prepare Tcl generated with prog_mask/probe_out5 alias
Server/API/GUI smoke	PASS	pytest targeted tests passed

Figure 1.1: 本轮验证摘要。40ch 未做真实 board-ready bitstream，因为当前仓库没有真实 40 路 XDC；这是保护逻辑，不是失败。

本轮最重要的结论如下：

- 4ch core simulation 通过，日志包含 `ZLC\_SIM\_PASS: 4ch pulse streamer upload/start/timing/done verified`。
- 4ch Vivado 实现产物存在：`zlc_pulse_streamer_top_4ch.bit`和`zlc_pulse_streamer_top_4ch.ltx`。timing report 显示 `All user specified timing constraints are met`，WNS 约为 `0.759 ns`。
- 40ch camera preset 可编译成完整 40-channel runtime program：40 channels、6 edges、final mask 为 0、trigger count 为 1。
- 40ch prepare dry-run 已生成 Vivado Tcl，VIO 写入路径仍然使用 `zlc_prog_mask/probe_out5`，

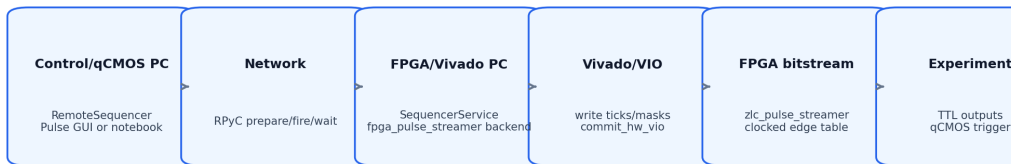
channel count 为 40。

- server/RemoteSequencer round trip 和 Pulse GUI 调用 attached sequencer 的 smoke test 已通过。
- 文档已把 Vivado 2019.2 硬编码改成 auto-detect/override 逻辑，同时说明本机 2019.1 与旧 address_switch 2019.2 的关系。

2

架构边界

Pulse GUI 不是硬件后端。它只负责显示、编辑、保存 pulse state, 并在按下 **Prepare/Fire/Wait/Safe** 时调用传入的 sequencer。真实硬件路径仍然是：



Only upload/start timing is host dependent. Pulse edges after start are executed by the FPGA clock.

Figure 2.1: 控制电脑、FPGA server、Vivado/VIO、FPGA bitstream 和 qCMOS trigger 的边界。

Command line

```
Control/qCMOS computer
RemoteSequencer.prepare/fire/wait_done
-> RPyC network
-> FPGA/Vivado computer SequencerService
-> fpga_pulse_streamer backend
-> Vivado/VIO writes ticks/masks
-> zlc_pulse_streamer FPGA clock executes edges
```

这个边界对实验安全很重要。Vivado、JTAG、网络 and Python 都只影响 prepare/start 的绝对时间；start 被 FPGA 接收以后，shot 内部的相对 edge timing 由 FPGA clock 推进。qCMOS acquisition 的安全顺序仍然是先 **prepare(sequence)**，再让 camera arm，最后 **fire()**。

3

Vivado 与路径策略

本地测试机器的 Vivado 是：

Command line

```
C:\Xilinx\Vivado\2019.1\bin\vivado.bat
```

旧 `references\source_archives\address_switch` 工程里的 VIO stub 显示它来自 Vivado 2019.2, 并且历史路径类似 `D:\time_sequence\address_switch`。因此新的 bat 不把 2019.1 或 2019.2 写死, 而是用 `vivado_env.bat` 搜索：

1. 如果已经设置 `ZLC_PS_VIVADO_BIN`, 优先使用它。
2. 查找 `C:\Xilinx \Vivado \*\bin \vivado .bat` 和 `D:\Xilinx \Vivado \*\bin \vivado .bat`。
3. 查找 PATH 里的 `vivado.bat` 或 `vivado`。

Vivado 2019 的 debug core 还有路径长度问题。本地深 Dropbox 路径曾触发 debug hub/VIO 生成失败；短路径或 `subst` 后可以通过。新的 `vivado_run_tcl.bat` 会尽量用临时短盘符运行 Tcl。去 Verilog 电脑时仍建议直接放在短路径，例如：

Command line

```
D:\ZLC
D:\time_sequence\ZLC
C:\ZLC
```

4

4ch 仿真证据

新增的可复现入口是：

Command line

```
.\fpga\pulse_streamer\simulate_4ch_core.bat
```

它会调用 `xvlog/xelab/xsim`, 编译 `zlc_pulse_streamer.v` 和 `tb_zlc_pulse_streamer_4ch.v`。testbench 上传的 edge table 是：

4ch simulation edges

tick 0	mask 0011
tick 3	mask 0001
tick 5	mask 1101
tick 6	mask 0101
tick 12	mask 0001
tick 13	mask 0000

仿真检查 `reset`、`prog_we` 写入、`start edge`、每个 tick 的输出 `mask`、`running/done` 和最终全 off。

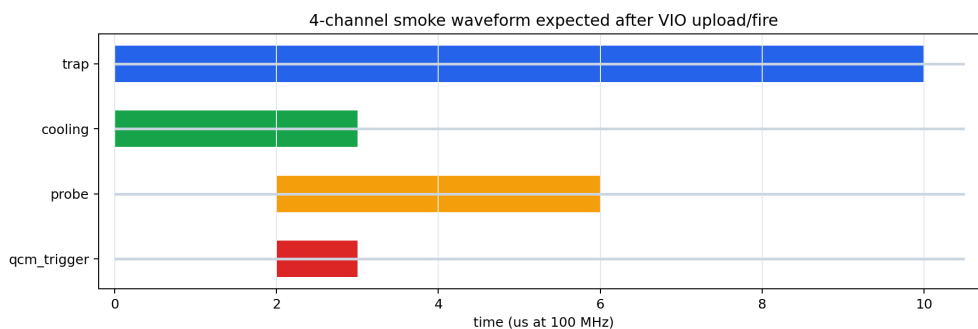


Figure 4.1: 4ch smoke waveform 的物理含义。仿真使用缩短 tick，真实 smoke test 在 100 MHz 下对应 us 级输出。

当前 xsim 结果:

xsim result

ZLC_SIM_START: 4ch pulse streamer core

ZLC_SIM_PASS: 4ch pulse streamer upload/start/timing/done verified

5

4ch Vivado build 证据

4ch first-light 使用历史 address_switch 的四个输出 pin:

4ch pin map

out[0]	-> trap	-> PACKAGE_PIN M17
out[1]	-> cooling	-> PACKAGE_PIN F15
out[2]	-> probe	-> PACKAGE_PIN N15
out[3]	-> qcm_trigger	-> PACKAGE_PIN R17
clk		-> PACKAGE_PIN R4
running LED		-> PACKAGE_PIN R2
done LED		-> PACKAGE_PIN R3

当前 build artifact:

Command line

```
C:\Users\eadri\Dropbox\WorkCode\Github\Zou_lab_control_v1\fpga\pulse_streamer_4ch\build\zlc_pulse_streamer_4ch\zlc_pulse_streamer_4ch.runs\impl_1\zlc_pulse_streamer_top_4ch.bit
C:\Users\eadri\Dropbox\WorkCode\Github\Zou_lab_control_v1\fpga\pulse_streamer_4ch\build\zlc_pulse_streamer_4ch\zlc_pulse_streamer_4ch.runs\impl_1\zlc_pulse_streamer_top_4ch.ltx
```

Timing evidence:

timing summary

All user specified timing constraints are met: True
WNS(ns): 0.759

4ch build/program 推荐流程:

Command line

```
cd D:\ZLC
.\fpga\pulse_streamer\vivado_env.bat
.\fpga\pulse_streamer\simulate_4ch_core.bat
.\fpga\pulse_streamer\build_4ch_bitstream.bat
.\fpga\pulse_streamer\program_4ch_fpga.bat
.\fpga\pulse_streamer\smoke_test_4ch_upload.bat
```

6

40ch 自查边界

40ch 的 HDL top 已经存在, VIO contract 中 `probe_out5` 的宽度是 40, 也就是 runtime program 的 mask 是 40 bits。当前仓库没有真实 `zlc_pulse_streamer_40ch.xdc`, 只有 `zlc_pulse_streamer_40ch.xdc.template`。因此真实 40ch bitstream build 和 program 应该停止, 直到每一个 `<PIN_CHxx>` 都被真实 package pin 替换。

允许在没有真实 XDC 前做的是:

Command line

```
.\fpga\pulse_streamer\check_40ch_synth.bat
```

这个入口只检查 40ch HDL/VIO 宽度, 不生成 board-ready bitstream。当前本轮按照用户要求没有实际跑完整 40ch build; 这里的自查重点是保证软件链路和文档流程不把没有 pin map 的设计误烧到硬件。

40ch camera preset 的编译结果:

camera_imaging_40ch runtime program

```
channels: 40 channels, ch00 ... ch39
ticks: 0, 200000, 210000, 212000, 2210000, 2212000
masks: 3, 1, 13, 5, 1, 0
trigger_count: 1
final_mask_zero: True
```

prepare dry-run 生成的 Tcl:

Command line

```
C:\Users\eadri\Dropbox\WorkCode\Github\Zou_lab_control_v1\fpga\pulse_streamer\build\verify_40ch\pulse_streamer_prepare.tcl
```

这个 Tcl 会查找 `zlc_prog_mask` 或 `probe_out5`, 并用 40-bit mask 写入 VIO。GUI 中隐藏 channel 不会改变 bit index; 隐藏只是视图状态。

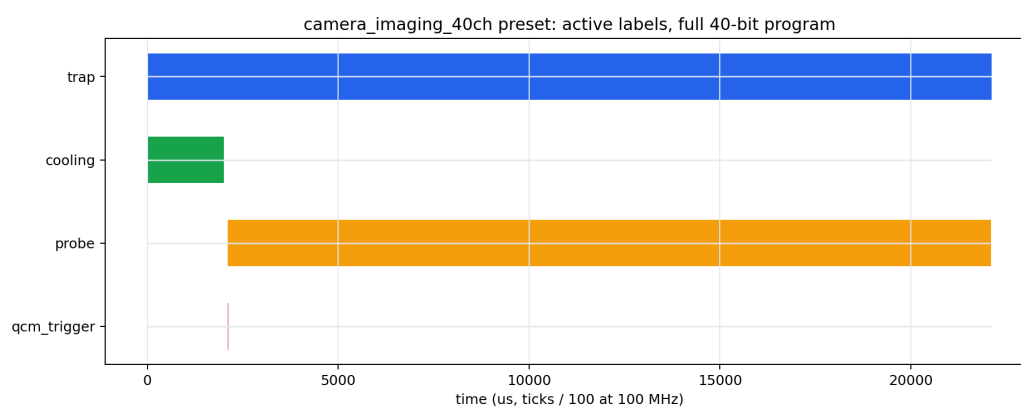


Figure 6.1: 40ch preset 中当前 active/display label 的四路。Runtime program 仍然保留完整 40-channel bit order。

7

Server 与 GUI 自查

server 的核心 contract 是 `SequencerService`: control 电脑传 `PulseSequence`, server 编译成 `RuntimeSequenceProgram`, 然后调用 command backend。已通过的 targeted tests 包括:

- `test_remote_sequencer_round_trip_uses_json_protocol`: 本地 RPyC server/client round trip, `prepare/fire/wait_done` 走 JSON protocol.
- `test_checked_in_camera_imaging_pulse_compiles_for_40ch_fpga`: 40ch pre-set 编译到 40-channel edge table.
- `test_pulse_gui_controls_call_attached_40ch_sequencer`: Pulse GUI 调用传入的 40ch sequencer, `editor.last_program.channels` 是完整 40ch.

Verilog/FPGA 电脑启动 4ch server:

Command line

```
.\fpga\pulse_streamer\start_server_4ch.bat
```

填完 40ch XDC、build 和 program 后启动 40ch server:

Command line

```
.\fpga\pulse_streamer\start_server_40ch.bat
```

在 Verilog 电脑本机开 Pulse GUI, 用第二个 terminal 连接本机 server:

Command line

```
.\pulse_gui.bat --remote-host 127.0.0.1 --remote-port 18861 --state  
↪ .\pulses\camera_imaging_40ch.json
```

在控制/qCMOS 电脑开 Pulse GUI, 把 host 换成 FPGA 电脑打印出来的 IP:

Command line

```
.\pulse_gui.bat --remote-host 192.168.x.y --remote-port 18861 --state  
↪ .\pulses\camera_imaging_40ch.json
```

GUI 的 channel name panel 左侧是硬件 channel，右侧是 display label。Preview y-axis 使用 display label；保存和编译仍然使用硬件 channel。40ch 默认只显示前四路，其它 channel 从下拉框添加；Hide Off 只影响视图，不改变 mask bit order。

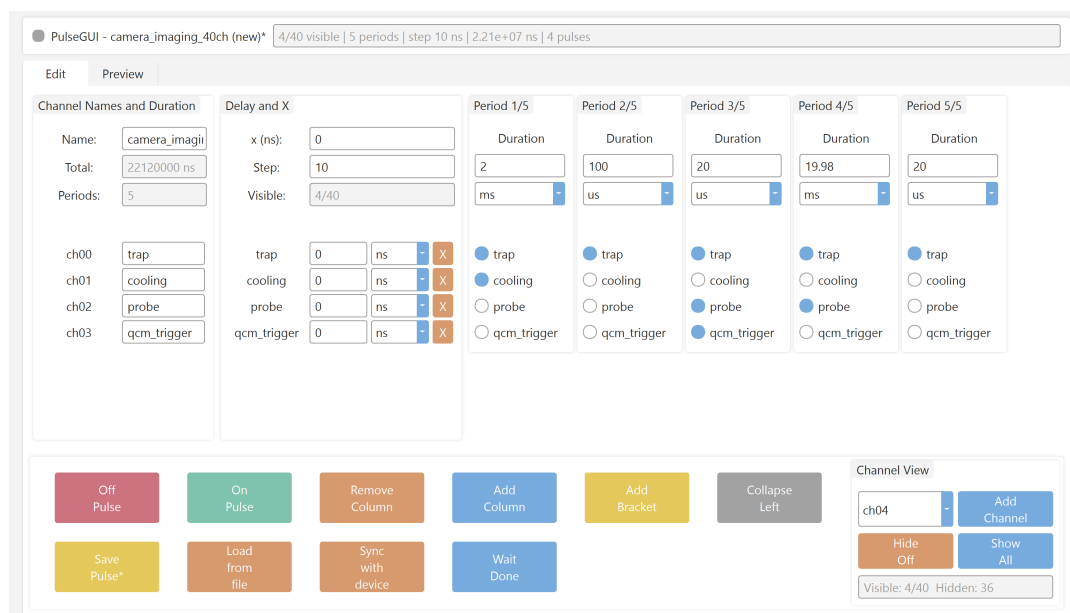


Figure 7.1: Pulse GUI 的 40-channel camera preset 编辑界面。GUI 是前端，真正的硬件动作由 attached sequencer/server 完成。

8

上机前 checklist

1. 在 Verilog 电脑把 repo 放到短路径, 例如 `D:\ZLC` 。
2. 运行 `install_requirements.bat` 或 `python -m pip install -e ..`。
3. 运行 `vivado_env.bat`, 确认 Vivado 路径正确; 必要时设置 `ZLC_PS_VIVADO_BIN`。
4. 运行 `simulate_4ch_core.bat`, 确认 xsim pass。
5. 运行 `build_4ch_bitstream.bat`, 确认 `.bit/.ltx` 生成。
6. 运行 `program_4ch_fpga.bat` 把 4ch bitstream 和 `.ltx` probes 烧进 FPGA; 40ch 完整 build 后对应运行 `program_40ch_fpga.bat`。
7. Program 成功后运行 `smoke_test_4ch_upload.bat`, 用示波器看四路波形。若已经手动在 Vivado GUI 中 program 了同一个 bit/probes, 或明确设置 `ZLC_PS_VIVADO_PROGRAM_ON_RUN="1"` 让 backend 第一次 run 时烧板, 可以不重复执行 program bat; 常规上机推荐显式运行 program bat。
8. 启动 `start_server_4ch.bat` 或 40ch 完整 build 后的 `start_server_40ch.bat`。
9. 在本机或控制电脑打开 `pulse_gui.bat -remote-host ...`, 按 Prepare 时观察 server state/log。
10. qCMOS 接入前确认 `qcm_trigger/ch03` 物理线、电平、触发极性和示波器波形。

9

故障定位

- 找不到 Vivado: 运行 `vivado_env.bat`, 或设置 `ZLC_PS_VIVADO_BIN`。
- Vivado debug core 路径报错: 把 repo 放到短路径, 或检查 `vivado_run_tcl.bat` 是否能使用临时 `subst`。
- 40ch build 提前停止: 检查是否缺少 `zlc_pulse_streamer_40ch.xdc`, 或是否仍有 `<PIN_CHxx>` placeholder。
- Prepare 失败: 看 server state dir 里的 `prepare.log` 和 `pulse_streamer_prepare.log`, 确认 `.xpr/.bit/.ltx` 存在、VIO probe 名匹配。
- GUI 能打开但硬件没动: 确认 GUI 是用 `-remote-host` 连到 server, 而不是离线 `RuntimeSequencer`; 看 server terminal 是否收到 `prepare/fire`。
- qCMOS timeout: 确认 external trigger 接的是 `qcm_trigger` 或 40ch 的 `ch03`, 并且 TTL 正沿到达相机。